

# Regular Expressions and Finite State Machines

In this session we'll introduce the notion of formal languages and consider a simple kind of language called *regular languages*. By the end of this session you should be familiar with

- describing these languages using *regular expressions*
- recognizing these languages using *finite state machines*

You may also come across finite state machines being called *finite state automata*. The Greek word *automata* is the plural of *automaton*, by which we just mean "machine".

and should be able to relate these two forms.

## Formal languages

General linguistics is hard. In computer science, we work with a simplified view of languages, called *formal languages*.

We are concerned here only with the surface structure (syntax) of languages, not their meaning (semantics), though surface structure is a necessary precursor to meaning.

### definition

An alphabet (usually written  $\Sigma$ ) is a finite set of symbols or letters.

Some example alphabets:

- $\Sigma_{\text{bits}} = \{0, 1\}$
- $\Sigma_{\text{digits}} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- $\Sigma_{\text{letters}} = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$

### definition

A string over an alphabet  $\Sigma$  is a finite sequence of symbols from the alphabet  $\Sigma$ .

- The set of strings over an alphabet  $\Sigma$  is written  $\Sigma^*$ .
- The empty string is usually written  $\epsilon$  (in some books,  $\lambda$ ).

The symbols of a string are usually written together, without spaces or commas. For example:

- Strings of bits:  $\epsilon$ , 0, 1, 01, 10110
- Strings of letters:  $\epsilon$ , a, b, bad, better

Notice that although each string has a finite length, there are infinitely many of them. That is,  $\Sigma^*$  is an infinite set.

definition

A language over an alphabet  $\Sigma$  is a finite or infinite set of strings over the alphabet, that is, a subset of  $\Sigma^*$ .

The following are examples of languages over the alphabet  $\Sigma = \{a, b\}$ :

- $\emptyset$ , the empty set of strings
- $\{\epsilon\}$ , the set containing only the empty string
- $\{\epsilon, a, b, aa, ab, ba, bb\}$ , the set of strings of length at most 2
- $\{\epsilon, a, aa, aaa, \dots\}$ , the infinite set of strings consisting of just a's
- $\{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$ , the infinite set of all combinations of a's and b's
- $\{\epsilon, ab, aabb, aaabbb, aaaabbbb, \dots\}$ , the infinite set of strings consisting of  $n$  a's followed by  $n$  b's.

Note the difference between  $\emptyset$  and  $\{\epsilon\}$ .

For any formal language, a key task is testing for membership of the set of strings.

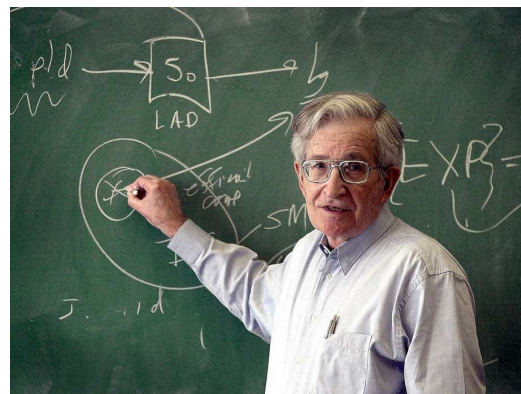
definition

A recognizer for a language  $L$  over an alphabet  $\Sigma$  is a machine that can be used to decide whether a string over  $\Sigma$  is in  $L$ .

## The Chomsky hierarchy

There are several ways to define particular formal languages. The following types were enumerated by the linguist Noam Chomsky. Each type is a restriction of the earlier ones.

| Type | Languages         | Descriptions               | Recognizers                    |
|------|-------------------|----------------------------|--------------------------------|
| 0    | decidable         | general rewriting systems  | Turing machines                |
| 1    | context-sensitive | context-sensitive grammars | linear-bounded Turing machines |
| 2    | context-free      | context-free grammars      | Pushdown machines              |
| 3    | regular           | regular expressions        | Finite state machines          |



Noam Chomsky revolutionized modern linguistics by applying insights from formal language theory.

Computer language processing relies on the last two, and indeed not all context-free languages. These languages are simple enough to allow efficient recognizers, that is, algorithms that recognize a string of length  $n$  in time  $O(n)$ . This week we'll consider regular expressions. Next week we'll consider the more powerful context-free grammars.

## Operations on formal languages

There are various operations we can use to make new languages from existing languages.

### definition

The union of two languages  $L_1$  and  $L_2$  over the same alphabet  $\Sigma$ , written  $L_1 \cup L_2$ , is just the union of the two sets of strings.

For example,

- $\{a, ab\} \cup \{a, bc\} = \{a, ab, bc\}$

### definition

The intersection of two languages  $L_1$  and  $L_2$  over the same alphabet  $\Sigma$ , written  $L_1 \cap L_2$ , is just the intersection of the two sets of strings.

For example,

- $\{a, ab\} \cap \{a, bc\} = \{a\}$

definition

The complement of a language  $L$  over the alphabet  $\Sigma$ , written  $\bar{L}$ , is  $\Sigma^* - L$ .

For example, if  $\Sigma = \{a\}$

- $\overline{\{\epsilon, a, aa\}} = \{aaa, aaaa, aaaaa, \dots\}$
- $\overline{\{\epsilon, aa, aaaa, \dots\}} = \{a, aaa, aaaaa, \dots\}$

definition

The concatenation of two languages  $L_1$  and  $L_2$  over the same alphabet  $\Sigma$ , written  $L_1L_2$ , is the set of strings formed by concatenating a string from  $L_1$  with one from  $L_2$ , i.e.

$$L_1L_2 = \{uv \mid u \in L_1 \wedge v \in L_2\}$$

For example,

- $\{a, bc\}\{de, f\} = \{ade, af, bcde, bcf\}$
- $\{\epsilon, a\}\{a, bc\} = \{a, bc, aa, abc\}$

definition

The Kleene closure of a language  $L$  over the alphabet  $\Sigma$ , written  $L^*$ , is the set of strings of the form  $w_1w_2\dots w_n$  where each  $w_i \in L$ , for  $n \geq 0$ . Another way of saying that is that  $L^*$  is the smallest set satisfying

$$L^* = \{\epsilon\} \cup LL^*$$

For example,

- $\{a\}^* = \{\epsilon, a, aa, aaa, aaaa, \dots\}$
- $\{a, b\}^* = \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$
- $\{a, bc\}^* = \{\epsilon, a, bc, aa, abc, bca, bcabc, \dots\}$
- $\{a, ab\}^* = \{\epsilon, a, aa, ab, aaa, aab, aba, aaaa, \dots\}$



Stephen Kleene was an American mathematician who studied recursion theory and formal languages

# Regular expressions

Regular expressions are a way of describing simple languages, based on a few of the above operations.

## definition

A regular expression over an alphabet  $\Sigma$  can be

|             |                                                                     |
|-------------|---------------------------------------------------------------------|
| $\emptyset$ | the empty language                                                  |
| $\epsilon$  | the language $\{ \epsilon \}$                                       |
| $s$         | the language $\{ s \}$ , where $s$ is a single symbol from $\Sigma$ |
| $R S$       | the union of the languages described by $R$ and $S$                 |
| $RS$        | the concatenation of the languages described by $R$ and $S$         |
| $R^*$       | the Kleene closure of the language described by $R$                 |
| $(R)$       | the language described by $R$                                       |

where  $R$  and  $S$  are regular expressions over the alphabet  $\Sigma$ .

Some books use minor variations on this notation:

- They may use the Greek letter  $\lambda$  instead of  $\epsilon$ .
- They may use  $\cup$  or  $+$  instead of  $|$ .

Some conventions allow us to save on parentheses:

- $(R|S)|T$  is the same as  $R|(S|T)$ , so the parentheses can be omitted.
- $(RS)T$  is the same as  $R(ST)$ , so the parentheses can be omitted.
- We assume  $*$  binds more tightly than concatenation, which binds more tightly than union, so instead of  $(a(b^*))|c$ , we just write  $ab^*|c$ . However parentheses would be required for  $(ab)^*|c$  or  $a(b^*|c)$ .

For example, using the alphabet  $\Sigma = \{a, b\}$ :

| Regular | Language |
|---------|----------|
|---------|----------|

expression

|           |                                                    |
|-----------|----------------------------------------------------|
| a         | { a }                                              |
| a ba      | { a, ba }                                          |
| (a bb)*   | { ε, a, bb, aa, abb, bba, bbbb, ... }              |
| a(a bb)*a | { aa, aaa, abba, aaaa, aabba, abbaa, abbbba, ... } |

Try some yourself. Type a regular expression in one of the boxes (they all do the same thing) and hit return to see (part of) the language it describes. Try to work out the general shape of strings in the language, and how it relates to the regular expression.

| Regular expression   | Language |
|----------------------|----------|
| <input type="text"/> |          |
| <input type="text"/> |          |
| <input type="text"/> |          |

Some things to try:

- $a(a|b)^*$
- $(a|b)^*a(a|b)^*$
- $((a|b)(a|b))^*$
- $(a|b)((a|b)(a|b))^*$
- $b^*ab^*$
- $b^*ab^*ab^*$

definition

A regular language is a language described by a regular expression.

Note that the same language may be described by more than one regular expression. For example,

- $a(b|c)d$  describes the same language as  $abd|acd$
- $a(ba)^*b$  describes the same language as  $ab(ab)^*$ :  
{ ab, abab, ababab, abababab, ... }  
strings consisting of one or more concatenations of ab.
- $(a^*b^*)^*$  and  $(b^*a^*)^*$  each describe the same language as  $(a|b)^*$

## Searching with extended regular expressions

Regular expressions are commonly used as a more powerful matching tool than plain strings. Many programming languages provide libraries implementing matching with regular expressions with extra features beyond basic regular expressions, e.g. [java.util.regex.Pattern](#) in Java.

Type a regular expression (using extended syntax) in the box, and hit return to perform a search in the box below.

Examples:

- capitalized words: `[A-Z][a-z]*`
- numbers: `[0-9]+`

Search pattern:

This is some text for trying out searching regular expressions.

Edit the search expression and hit Return to try searching for: words, numbers, capitalized hostnames, URLs, e-mail addresses and postal addresses.

Miscellaneous text to practice on:

Google, 1-13 St Giles High Street, London WC2N 7031 3000 <https://www.google.co.uk/>

BBC Television Centre 020 8743 8000  
101 Wood Lane, Shepherd's Bush, London W12 8AP  
<https://televisioncentre.com/>

Martin Nyx Brain [martin.brain@city.ac.uk](mailto:martin.brain@city.ac.uk) -  
Department of Computer Science [ug.cs@city.ac.uk](mailto:ug.cs@city.ac.uk)  
<https://www.city.ac.uk/about/schools/science>  
School of Science and Technology  
<https://www.city.ac.uk/about/schools/science>  
City, University of London <http://www.city.ac.uk>  
Northampton Square, London EC1V 0HB 020 7041 3044

Common shorthands supported by many regular expression libraries:

|                     |                                                                                            |
|---------------------|--------------------------------------------------------------------------------------------|
| <code>[abc]</code>  | short for <code>(a b c)</code> ; character ranges like <code>a-z</code> are also supported |
| <code>[^abc]</code> | any single character except <code>a</code> , <code>b</code> or <code>c</code>              |
| <code>.</code>      | any single character                                                                       |
| <code>R?</code>     | short for <code>( R)</code> , i.e. zero or one <code>R</code>                              |
| <code>R+</code>     | short for <code>RR*</code> , i.e. one or more <code>R</code> 's                            |
| <code>\c</code>     | literal instance of a special character (like <code>.</code> or <code>*</code> )           |

Descriptions of the syntax of programming languages are typically down into two parts. Regular expressions are used to specify the basic symbols of the language, e.g.

depth in the Language Processors module next year.

|                             |                |
|-----------------------------|----------------|
| $[A-Za-z\_][A-Za-z0-9\_]^*$ | identifier     |
| <hr/>                       |                |
| $0 [1-9][0-9]^*$            | decimal number |

The rules for combining these basic symbols are usually specified using context-free grammars (discussed next week).

## Properties of regular languages

- All finite languages are regular (why?)
- The union or concatenation of two regular languages is regular, and the Kleene closure of a regular language is regular (a trivial consequence of the definition). That is, regular languages are closed under union, concatenation and Kleene closure.
- Less obviously, regular languages are closed under intersection and complement.
- Not all languages are regular, e.g. the infinite set of strings  $\{a^n b^n \mid n \geq 0\}$  consisting of  $n$  a's followed by  $n$  b's. (See later.)

## Finite State Machines

A finite state machine is a simple machine with which we can decide whether a string belongs to a particular regular language.

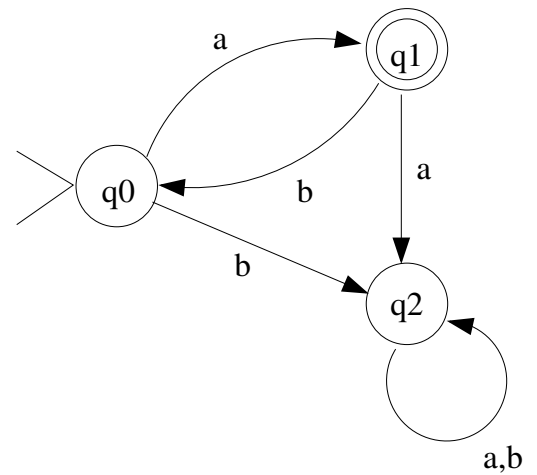
For example, take the alphabet  $\Sigma = \{a, b\}$  and consider the regular expression  $a(ba)^*$ , which describes the language  $\{a, aba, ababa, abababa, \dots\}$ .

The finite state machine at right is a recognizer for this language. It consists of

- Three states  $q_0, q_1$  and  $q_3$  (as implied by the name, the number of states must be finite)

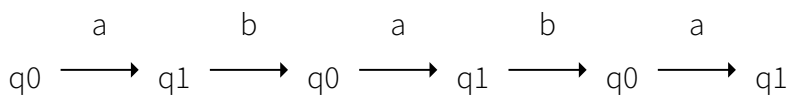


- Exactly one of these ( $q_0$ ) has a large incoming arrowhead, indicating that this is the start state.
- Some states (in this case just  $q_1$ ) have double circles, indicating that they are accept states. In general, there can be more than one accept state, and the start state can also be an accept state.
- For each state ( $q_0$ ,  $q_1$  or  $q_2$ ), and each symbol in the alphabet (a or b), there is an arrow from that state to another state, labelled with the symbol, called a transition. In some cases the transition may be back to the same state. When the transitions for both a and b go to the same place, we use a shorthand with a single arrow labelled a,b.



*Finite state machine recognizing  $a(ba)^*$ .*

Now let's use this machine to scan the string *ababa*. We start the machine in the start state  $q_0$ , and consider each symbol in turn, following the corresponding transition to a new state, until we reach the end of the string:

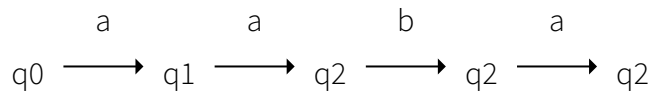


That is:

- Start in state  $q_0$  (the start state).
- Read a, and follow the transition to state  $q_1$ .
- Read b, and follow the transition to state  $q_0$ .
- Read a, and follow the transition to state  $q_1$ .
- Read b, and follow the transition to state  $q_0$ .
- Read a, and follow the transition to state  $q_1$ .
- Now we have reached the end of the input, and we are in an accept state ( $q_1$ ), so we accept the input as a member of the language.

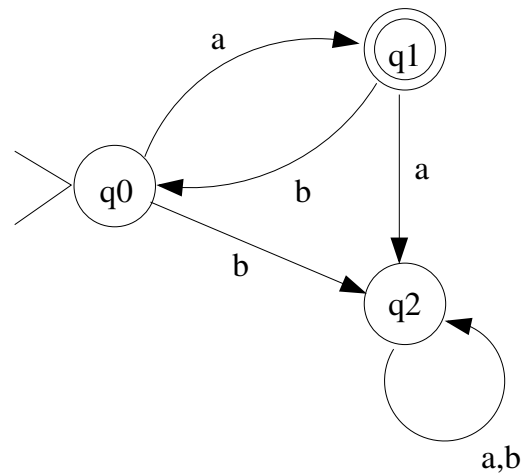
If we had been scanning the string *abab* instead, we would have ended up in state  $q_0$ , which is not an accept state, so we would have rejected the input.

Let's try that again with a different input, *aaba*:



Test your understanding by working out which state the following strings would end up in (starting from the start state in each case), and whether the string would be accepted:

- abba
- aba
- abaab



## FSMs and regular expressions

definition

Fact: Any language recognized by a finite state machine can be described by a regular expression (i.e. is a regular language), and vice versa.

As a consequence, we can decide whether a string of length  $n$  belongs to a particular regular language using  $O(n)$  time and  $O(1)$  space.

There are algorithms to convert in both directions, but we'll leave those to the Theory of Computation module and be more informal here.

One way of designing an FSM for a regular expression is to choose states to remember certain inputs, distinguishing those we want from those we don't.

### Example: counting

Let's design a finite state machine for the language of strings over the alphabet  $\Sigma = \{a, b\}$  containing exactly two  $b$ 's, i.e. the language described by the regular expression  $a^*ba^*ba^*$ .

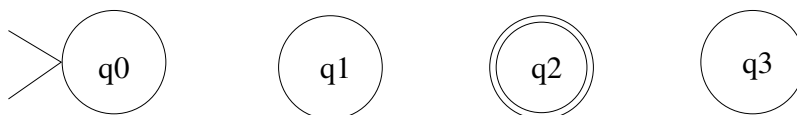
Try this regular expression:

We can use states to keep track of how many b's we've seen:

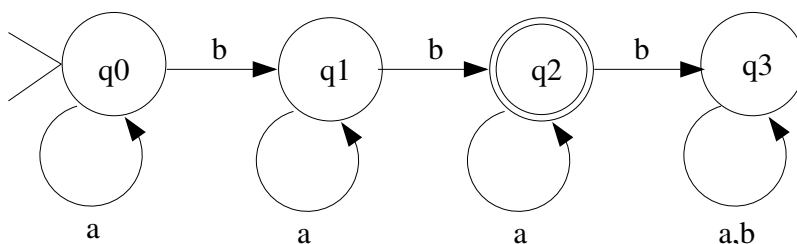
|    |                                                |                       |
|----|------------------------------------------------|-----------------------|
| q0 | not seen any b's (but may have seen a's)       | $a^*$                 |
| q1 | seen one b (and any number of a's)             | $a^*ba^*$             |
| q2 | seen two b's (and any number of a's)           | $a^*ba^*ba^*$         |
| q3 | seen three or more b's (and any number of a's) | $a^*ba^*ba^*b(a b)^*$ |



Since q0 includes the empty string  $\epsilon$ , that is our start state. q2 contains exactly the strings of the language, so we make it an accept state. (In general, there may be more than one.)



We can also work out the transitions from the strings seen by each state. For example, if we're in state q1, we've seen  $a^*ba^*$ . If the next symbol is a, that is still  $a^*ba^*$ , so we stay in state q1. If the next symbol is b, that will take us to  $a^*ba^*ba^*$ , i.e. state q2.



Try scanning some strings with this machine:

- aaa
- abaa
- ababaa
- bababbb

## Another example

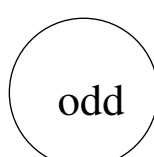
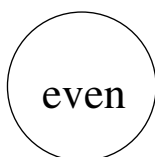
Let's try another language, the set of strings over the alphabet  $\Sigma = \{a, b\}$  containing an odd number of a's and any number of b's. There are several regular expressions describing this language, including:

- $(b^*ab^*a)^*b^*ab^*$
- $b^*ab^*(ab^*ab^*)^*$
- $b^*a(b^*ab^*a)^*b^*$

Try the above expressions to check that they're equivalent. See if you can come up with another expression that is equivalent to these.

| Regular expression   | Language |
|----------------------|----------|
| <input type="text"/> |          |
| <input type="text"/> |          |

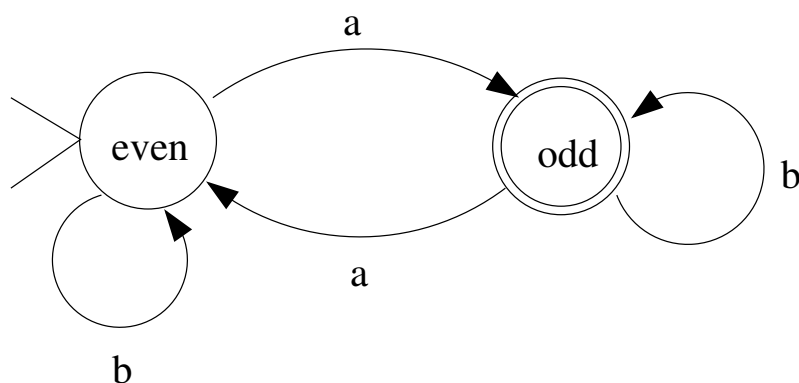
In order to decide whether to accept an input, our machine will need to keep track of whether the number of a's seen so far is even or odd. We can do this with two states, which we label "even" and "odd".



Now when the machine starts, we've seen zero a's, which is an even number, so we should make "even" our start state. We will accept the string if we've seen an odd number of a's, so we make "odd" an accept state.



It remains to fill in the transitions. An input of a will move us from the even state to the odd one, and vice versa. An input of b will leave us the current state unchanged:



Test your understanding by working out the sequence of states visited in scanning each of these strings with this machine:

- bab
- abbab
- aaaba

In each case, do you finish in an accept state?

## Limitations of regular languages

The fact that a regular language must be recognized by a finite state machine enables us to explore the limitations this implies. We saw above that a finite state machine can keep track of whether it has seen an odd or an even number of a's. It can even keep track of whether it has seen a multiple of 10 a's. We've also seen that a finite state machine can count up to a fixed number.

But it cannot record whether it has seen more a's than b's, because to do that it would need to keep track of how many more it had seen, and this unbounded number cannot be recorded with finitely many states. Thus such a language is

Balanced parentheses (), brackets [], and braces {}, and pairings like if-else and do-while, are a very important feature of programming languages. They require something more powerful than regular expressions, namely the context-free grammars we'll cover next week.

not regular.

Similarly, a language of matched brackets, like  $\{ \epsilon, ab, aabb, aaabbb, \dots \}$  is not regular, because no finite state machine can remember an arbitrary number of a's.

Similarly, a language of palindromes (words that are the same read forward or backward, like "madam") cannot be regular.

Next time we'll look at a more general class of languages, which includes these examples.

---